

SQL Joins

 [_ \(https://github.com/learn-co-curriculum/python-p3-sql-joins\)](https://github.com/learn-co-curriculum/python-p3-sql-joins)  [_ \(https://github.com/learn-co-curriculum/python-p3-sql-joins/issues/new\)](https://github.com/learn-co-curriculum/python-p3-sql-joins/issues/new)

Learning Goals

- Use primary and foreign keys to create the relations in a relational database.
- Use **INNER JOIN** and **LEFT JOIN** to retrieve relevant data for members of one table in other tables.

Key Vocab

- **Primary Key**: a number that uniquely identifies one record in a table.
- **Foreign Key**: a column or group of columns that connects one table to another.
- **Join**: a query that returns related records from multiple tables in a single record.
- **One-to-Many**: a type of relationship between tables where one record in table A is connected to multiple records in table B. **e.g.** One person ordering multiple drinks at a bar.
- **Many-to-Many**: a type of relationship between tables where multiple records in table A are connected to multiple records in table B. **e.g.** Students have many classes and classes have many students.

Introduction

We'll discuss how to retrieve specific sets of data from associated tables using SQL join statements.

[Fork and clone this lesson](https://github.com/learn-co-curriculum/python-p3-sql-joins/fork)  [_ \(https://github.com/learn-co-curriculum/python-p3-sql-joins/fork\)](https://github.com/learn-co-curriculum/python-p3-sql-joins/fork) to code along.

What Is a JOIN?

A SQL **JOIN** clause is a way to combine rows from two or more tables, based on a common column between them. The great thing about relational databases is that they are just that—*relational*. Relational databases allow us not only to store data that is interconnected, but to retrieve that data in ways that reflect that interconnectivity.

Let's say, for example, we have two tables, a **cats** table and an **owners** table. Cats and owners are associated by a **foreign key** of **owner_id** in the **cats** table. How would we craft a query that would grab us all of the cats with a particular owner, and even include information about that owner in the data returned to us by that query?

We know how to write a **SELECT** statement that gets us all of the cats with a particular **owner_id**. For example:

```
SELECT * FROM cats WHERE owner_id = 2;
```

This would return us the appropriate list of cats. But what if we wanted to query *both* the **cats** and the **owners** tables and return information about both cats and owners? This is where **JOIN** statements come in.

JOIN Types

There are two common types of joins that we will cover in this lesson. The following JOIN keywords will be used along with **SELECT** statements to achieve the described return values.

Type	Description
INNER JOIN	Returns all rows when there is at least one match in BOTH tables
LEFT (OUTER) JOIN	Returns all rows from the left table, and the matched rows from the right table

In the following code-along, we'll be crafting JOIN statements of both of the above types in order to return data about a series of connected database tables.

For this code-along, we'll be working with the database of pets and owners that we created in the last exercise. It's available here in the **pets_database.db** file.

For the rest of this code along, you can run the SQL commands in your terminal from the `sqlite3` prompt, or you can open the `pets_database.db` file in DB Browser for SQLite, and run the SQL commands from the "Execute SQL" tab.

A Reminder on Foreign Keys

Note that the `cats` table has an `owner_id` column. This column is a **foreign key** that connects each cat to an individual owner. If an individual cat has an `owner_id` of `2`, that indicates that that cat belongs to the owner who has an `id` of `2`.

To confirm this, you can run the following `SELECT` statement:

```
SELECT * FROM cats WHERE owner_id = 2;
```

You should see just one cat returned to us, the one that belongs to Sophie, our second owner:

id	name	age	breed	owner_id
3	Nona	4	Tortoiseshell	2

NOTE: To output your query results in this format in the `sqlite3` prompt, enter the following settings before running your query:

```
sqlite> .headers on
sqlite> .mode column
sqlite> .width auto
```

Code Along I: INNER JOIN

Definition

An **INNER JOIN** query will return *all* the rows from both tables you are querying where a certain condition is met. In other words, **INNER JOIN** will select all rows from both tables as long as there is a match between the specified columns of each table.

Let's take a look at a boiler-plate **INNER JOIN** statement, before we try it out on our **pets_database** :

```
SELECT column_names
FROM first_table
INNER JOIN second_table
ON first_table.column_name = second_table.column_name;
```

This may not make sense to you just yet. Let's try it out with our own database in order to gain a better understanding.

Writing INNER JOINS

Let's write an INNER JOIN query that will return the name and breed of the cat along with the name of that cat's owner:

```
SELECT cats.name, cats.breed, owners.name
FROM cats
INNER JOIN owners
ON cats.owner_id = owners.id;
```

Let's break this down:

```
SELECT cats.name, cats.breed, owners.name
```

Here, we are specifying which columns from each table we want to select data from. We use the **table_name.column_name** notation to grab columns from two different tables.

Next up, we join our two tables together with our **INNER JOIN** keyword:

```
FROM cats
INNER JOIN owners
```

Lastly, we tell our query *how* to connect, or join, the two tables. In other words, we tell our query which columns in each table function as the foreign key/primary key. Through this, our query will correctly identify which cat belongs to which owner and return that information accordingly.

```
ON cats.owner_id = owners.id;
```

Here, we are telling our query that the `owner_id` column on the Cats table is filled with data that corresponds to data in the `id` column of the Owners table. We are indicating that a value of `1`, for example, in an individual cat's `owner_id` column refers to the individual owner who has an `id` of `1`. And we are telling our query to return *only* those cats and owners who share this connection.

The above statement should return the following:

name	breed	name
-----	-----	-----
Maru	Scottish Fold	mugumogu
Hana	Tabby	mugumogu
Nona	Tortoiseshell	Sophie

We did it! We wrote an `INNER JOIN` query that returns to us all of the data in the specified columns from both tables.

Notice that the owner's name column is called `name` in the output above. That is because we requested the `name` column from the `owners` table. For this particular output though, it would be great if the column could read "owners_name", to distinguish it from the cat's name column.

Let's run that query again, this time aliasing the `name` column of the `owners` table as `owners_name`, using the `AS` keyword:

```
SELECT
  cats.name,
  cats.breed,
  owners.name AS "owner_name"
FROM cats
INNER JOIN owners
ON cats.owner_id = owners.id;
```

This should return:

name	breed	owner_name
-----	-----	-----

Maru	Scottish Fold	mugumogu
Hana	Tabby	mugumogu
Nona	Tortoiseshell	Sophie

You can visualize what's happening with an **INNER JOIN** like this:

When using an **INNER JOIN** to return data from multiple tables, you will only see the results from records in the first table (left side of the diagram above) and the second table (right side) when they can be connected via a foreign key; that is, when our **cats** table has an **owner_id** that corresponds with an **id** in the **owners** table.

A Note on INNER JOINS, or, Where's Lil' Bub?

When we say that an **INNER JOIN** returns all of the data for which a certain condition is true, we mean that any data that does not meet a **JOIN** condition will not be returned. The **JOIN** condition, in this case, is the thing that our two tables are joined **ON** :

```
ON cats.owner_id = owners.id;
```

Our query, therefore, will select all of the appropriate data concerning cats and owners who *are* joined by an **owner_id / id** foreign key/primary key relationship. In other words, it will select all of the cats who have a value in the **owner_id** column that matches a value in the **id** column of the **owners** table. Any cats that have an empty **owner_id** column, or have a value in that column that does not match the **id** of an existing owner, will not be selected by the query.

You might have noticed that the data returned by our query did not include Lil' Bub. That's because when we inserted Lil' Bub into our **cats** table, we didn't give her an **owner_id** :

id	name	age	breed	owner_id
-----	-----	-----	-----	-----
1	Maru	3	Scottish Fold	1
2	Hana	1	Tabby	1

3	Nona	4	Tortoiseshell	2
4	Lil' Bub	2	perma-kitten	

Other types of **JOIN** statements, however, can return such data.

Code Along 2: LEFT JOIN

I don't know about you, but I miss Lil' Bub. It would be nice if we could query our database for both cat and owner information *without* excluding her. With a **LEFT JOIN** we can do just that.

LEFT JOIN Definition

A **LEFT JOIN** query returns *all* rows from the left, or first, table, regardless of whether or not they met the **JOIN** condition. The query will also return the matched data from the right, or second, table.

In the case of data from the first table that doesn't meet our **JOIN** condition, the resulting output will include **NULL**, or empty, values for the missing matched columns.

Let's take a look at a boiler-plate **LEFT JOIN** :

```
SELECT column_name(s)
FROM first_table
LEFT JOIN second_table
ON first_table.column_name = second_table.column_name;
```

Now let's try it out on our **pets_database** .

Writing **LEFT JOIN** s

Execute the following command:

```
SELECT cats.name, cats.breed, owners.name
FROM cats
LEFT OUTER JOIN owners
ON cats.owner_id = owners.id;
```

You should see the following output returned to you:

name	breed	name
-----	-----	-----
Maru	Scottish Fold	mugumogu
Hana	Tabby	mugumogu
Nona	Tortoiseshell	Sophie
Lil' Bub	perma-kitten	

Here, our **LEFT JOIN** has returned to us *all* of the cats (including Lil' Bub!), with matched data regarding owner's name for those cats that have an owner, and empty space in the owner's name column for the cat that doesn't have an owner.

Here's a visualization of a **LEFT JOIN** :

As you can see, with a left join, all the data is returned from the first table *regardless* of whether there's a foreign key match in the second table. We get back all the data from the **cats** data, and we get data from the **owners** table when our **cats** table has an **owner_id** that corresponds with an **id** in the **owners** table.

Other **JOIN** Types

Some SQL databases support two other types of joins: **RIGHT JOIN** and **FULL JOIN** . These are far less common than **INNER JOIN** and **LEFT JOIN** — in fact, SQLite doesn't support these types of joins, and it's very rare that you will need to use them. You can read more about these other join types [here](https://www.w3schools.com/sql/sql_join.asp) , if you're curious!

Conclusion

With **INNER JOIN** and **LEFT JOIN**, along with our knowledge of primary keys and foreign keys, we now have all the tools at our disposal to access data across related tables using SQL! These concepts will also translate back to our object oriented Python code once we start using SQLAlchemy, and knowing how tables relate is essential when it comes to creating the different tables you will need to effectively store data for your applications.

Resources

- [Difference between Primary Key and Foreign Key - GeeksforGeeks](https://www.geeksforgeeks.org/difference-between-primary-key-and-foreign-key/) ➞ (https://www.geeksforgeeks.org/difference-between-primary-key-and-foreign-key/)
- [SQL Joins - W3Schools](https://www.w3schools.com/sql/sql_join.asp) ➞ (https://www.w3schools.com/sql/sql_join.asp)
- [Practice SQL Queries on SQLBolt](http://sqlbolt.com/lesson/select_queries_review) ➞ (http://sqlbolt.com/lesson/select_queries_review)